EXPERIMENT 3: APPLY DECISION TREE AND RANDOM FOREST FOR CLASSIFICATION TASKS

1. DATASET SOURCE

- **Dataset Name:** Telco Customer Churn Dataset (Telco-Customer-Churn.xls)
- **Source:** This is a well-known dataset originally provided by IBM. It is publicly accessible via various data repositories, including Kaggle: Telco Customer Churn on Kaggle

2. DATASET DESCRIPTION

The Telco Customer Churn dataset contains data on a fictitious telecommunications company that provided home phone and internet services to 7,043 customers in California. It is widely used to predict whether a customer will churn (leave the service) based on their demographics, account information, and service usage.

- **Target Variable:** Churn (Binary: 1 for Yes / 0 for No).
- **Features:** The dataset comprises 21 columns, which include:
  - **Demographic Data:** gender, SeniorCitizen, Partner, Dependents.
  - **Services Subscribed:** PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies.
  - **Account Information:** tenure (number of months with the company), Contract (Month-to-month, One year, Two year), PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges.
- **Size:** 7,043 instances and 21 attributes.
- **Characteristics:** The dataset contains a mix of numeric (e.g., tenure, MonthlyCharges) and categorical features. It also exhibits a mild class imbalance, with significantly more non-churners than churners.

3. MATHEMATICAL FORMULATION OF THE ALGORITHMS

Decision Tree Classifier

A Decision Tree splits the dataset into subsets based on the feature that results in the highest information gain or the lowest impurity. The algorithm evaluates splits using **Gini Impurity** or **Entropy**.

For a node containing a dataset with C classes, where pi is the probability of an item belonging to class i, the Gini Impurity is defined as:

$$\text{Gini}(p) = 1 - \sum_{i=1}^{C} p_i^2$$

Alternatively, Information Gain is calculated using Entropy:

$$\text{Entropy}(S) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

The tree algorithms aim to maximize the Information Gain (IG) from a split:

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Random Forest Classifier

Random Forest is an ensemble learning method that constructs B multiple decision trees during training and outputs the mode of the classes for classification. It relies on **Bagging** (Bootstrap Aggregating) and random feature selection. Given a training set X=x1,…,xn with responses Y=y1 ,…,yn, bagging repeatedly selects a random sample with replacement and fits trees to these samples. The final prediction for an unseen sample x' is made by a majority vote of the B individual tree predictions hb(x):

$$\hat{y} = \text{mode}\{h_1(x'), h_2(x'), \dots, h_B(x')\}$$

4. ALGORITHM LIMITATIONS

- **Decision Tree Limitations:** * **Overfitting:** Trees that grow extremely deep tend to memorize the training data, leading to poor generalization on test data (high variance).
    - **Instability:** Small variations in the dataset can result in a completely different tree structure.
    - **Imbalance Sensitivity:** They tend to create biased trees if some classes dominate.

- **Random Forest Limitations:**
    - **Computational Cost:** Training hundreds of deep trees requires significant memory and computational power, which can be slow on very large datasets.
    - **Lack of Interpretability:** Unlike a single Decision Tree, which can be easily visualized and understood by humans, a Random Forest acts as a "black box" where individual feature paths are obscured by the ensemble.

5. METHODOLOGY / WORKFLOW

1. **Data Loading:** The dataset was loaded using the pandas library from an .xls file.
2. **Data Cleaning & Preprocessing:** The TotalCharges column, which contained hidden blank spaces, was coerced to numeric data (replacing empty strings with NaNs).
    - Rows with missing values (NaNs) were dropped.

    - The customerID column was removed as it serves no predictive value.
3. **Encoding:** Categorical variables were converted to numeric formats using LabelEncoder to make them compatible with Scikit-Learn's tree-based algorithms.
4. **Data Splitting:** The data was divided into feature space (X) and target variable (y), and then split into training (80%) and testing (20%) sets using train_test_split.
5. **Model Training:** A single DecisionTreeClassifier was initialized and fitted to the training set.
    - A RandomForestClassifier (with n_estimators=100) was initialized and fitted.
6. **Evaluation:** Predictions were made on the test set, and models were evaluated using Accuracy, Confusion Matrices, and Classification Reports (Precision, Recall, F1-score). Visualizations of confusion matrices and accuracy comparisons were plotted using seaborn and matplotlib.

6. PERFORMANCE ANALYSIS

Based on the execution outputs in the notebook:

- **Decision Tree Performance:**
    - **Accuracy:** 72.49%
    - **Class 1 (Churn) F1-Score:** 0.50 (Precision: 0.48, Recall: 0.52)
    - **Analysis:** The Decision Tree performs moderately but struggles distinctly with precision for the minority class (Churners). Its high variance likely caused it to overfit the training data, leading to a higher rate of false positives on the test set.
- **Random Forest Performance:**
    - **Accuracy:** 79.25%
    - **Class 1 (Churn) F1-Score:** 0.56 (Precision: 0.64, Recall: 0.49)
    - **Analysis:** The Random Forest significantly outperformed the single Decision Tree. The ensemble approach successfully lowered the variance, resulting in an ~7% increase in overall accuracy. Notably, the precision for detecting churners increased from 0.48 to 0.64, meaning when the Random Forest predicts a customer will churn, it is much more likely to be correct.

7. Hyperparameter Tuning

Tuning Process: Hyperparameter optimization is typically performed using GridSearchCV or RandomizedSearchCV combined with k-fold cross-validation (e.g., 5-fold) to systematically test different combinations of parameters.

- **Decision Tree Tuning:**
  - **Parameters Tuned:** max_depth (e.g., 5, 10, 15), min_samples_split (e.g., 2, 5, 10), and criterion (Gini vs. Entropy).
  - **Impact:** By capping the max_depth and increasing min_samples_split, the tree is prevented from building nodes that represent single outliers. This directly combats the Decision Tree's primary limitation (overfitting), resulting in a slight drop in training accuracy but an increase in test accuracy and better generalization.
- **Random Forest Tuning:**
  - **Parameters Tuned:** n_estimators (number of trees: 50, 100, 200), max_depth (e.g., 10, 20, None), and max_features ('sqrt', 'log2').
  - **Impact:** Tuning max_depth restricts individual tree complexity, while tuning n_estimators finds the sweet spot where adding more trees stops improving the F1-score, thereby saving computational resources. Tuning max_features ensures that the trees are sufficiently de-correlated. Proper tuning on this dataset typically yields a 1-3% boost in overall accuracy and balances the Precision-Recall trade-off for the churn class.

CONCLUSION

In conclusion, this experiment successfully demonstrated the application of tree-based machine learning algorithms to predict telecommunications customer churn. The comparative analysis clearly highlighted the predictive superiority of the Random Forest ensemble method over a single Decision Tree. By mitigating the high variance and overfitting inherent in individual trees, the Random Forest model achieved a higher overall accuracy (79.25%) and significantly better precision when identifying actual churners. While these baseline models provided a strong foundation, applying systematic hyperparameter tuning and handling the mild class imbalance would be the logical next steps to further optimize the model's performance for real-world business applications.